

MORE PROGRAMS OR MORE ROLLS? SEPARATING COVERAGE FROM SPECIALIZATION IN LLM HARNESSES

Anonymous authors

Paper under double-blind review

ABSTRACT

When a population of LLM harnesses solves more tasks than its best member, does the gain reflect program specialization or simply more attempts? We distinguish coverage observed after execution, task advantages that persist across repeats, and gains realized by a selector. The decisive control runs identical copies of one program: on MATH-500, nine single-call slots produce a positive repeat-averaged oracle estimate of 2.09 %, despite having no program specialization. Eight generated harnesses plus the baseline yield 2.38 % on their own complete-task set. Holding out repeats and calibrating against clones gives a mean gain of -0.10 % on 342 paired tasks; incomplete coverage and low sensitivity leave stable specialization unresolved. A separately frozen selector chooses the baseline on every task and gains 0.00 %. These results identify an attribution problem: positive population coverage does not establish task specialization, and coverage opportunities require a separate test of usable selection. We provide a controlled evaluation that keeps these judgments distinct. Gains are absolute probability differences expressed as %.

1 INTRODUCTION

Should a fixed inference budget buy more programs or more executions of one program? Automated agent design makes the first option increasingly accessible: a builder writes a population of LLM harnesses, each specifying prompts, tools, and control flow around a frozen solver (Hu et al., 2024; Nie et al., 2026; Zhang et al., 2026b). The appeal is that different programs might handle different tasks better. To justify that interpretation, however, a population’s gains must be separated from the extra attempts used to evaluate it.

The difficulty is that different observed successes do not require different abilities. Two independent executions of a program with success probability 0.5 cover a task with probability 0.75. No program specialization has been introduced. Pass@ k and self-consistency already exploit the value of additional sampling (Chen et al., 2021; Wang et al., 2022; Li et al., 2024). An oracle that selects a correct answer after seeing all outcomes can therefore show a population gain even when every member has the same code. Averaging a few repeats does not automatically remove this ambiguity: choosing the largest estimated success rate also favors sampling noise.

We ask what evidence distinguishes this coverage opportunity from task specialization. Three questions organize the study. Did the recorded executions contain additional correct answers? Does a member’s advantage on a task persist in a fresh execution? Can a rule choose usefully before seeing the answer, and improve on spending the same resources on a fixed program? These questions concern different quantities. A positive answer to the first need not answer the other two.

We first report single-run coverage in the full population of 35 generated MATH-500 programs, then test the specialization interpretation on a selected subset. The central control holds population size fixed and changes whether the programs differ: eight generated harnesses plus single-call are compared with nine byte-identical baseline slots. Three fresh repeats let us discover task-wise choices on two executions and score them on the third. The same-code arm measures what this procedure reports without program specialization. On the same panel, a selector frozen on development tasks then tests selection from question features, and an execution-matched oracle replay compares avail-

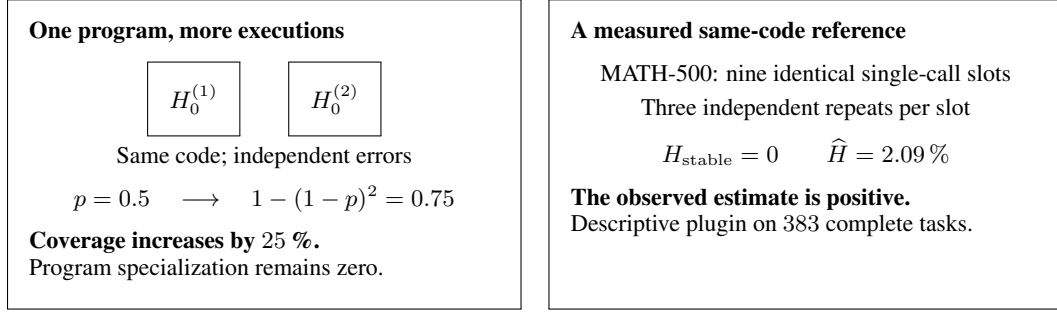


Figure 1: **More coverage does not identify more specialization.** Left: a probability calculation for two independent executions of one program. Right: the empirical clone arm produces a positive repeat-averaged oracle estimate despite zero program specialization (complete-task estimate; §5.2). The panels illustrate different quantities, not matched effect sizes. Our control design compares eight generated programs plus single-call with nine identical single-call slots, then separates repeat validation from feature-based pre-execution selection.

able coverage. Finally, BIRD traces examine whether code differences actually reach execution. They provide a separate diagnosis, not a causal explanation of the MATH comparison.

The evidence yields three contributions. **First**, the same-code control produces a positive repeat-averaged oracle estimate, establishing that this signal alone cannot identify program specialization. **Second**, held-out repeats and calibration show where the current data stop: the clone-adjusted mean is -0.10% , but missingness and limited power leave stable specialization unresolved. **Third**, separating selection from coverage changes the practical assessment: the tested selector gains 0.00% , and the full execution-matched replay favors additional single-call executions in its observed oracle mean. These findings separate coverage, repeat evidence, and realized selection.

2 RELATED WORK

Automated agent and harness design. ADAS treats agent programs as a search space (Hu et al., 2024). TTHe, Self-Harness, JIT-Agent, and Task-CoEvolve generate or adapt scaffolding around a frozen model (Nie et al., 2026; Zhang et al., 2026b;a; Miyai et al., 2026). HarnessLens, HarnessFix, AHE, HarnessBank, the Harness Handbook, and HarnessForge examine behavior through rollout allocation, traces, prediction checks, descriptors, localization, or joint policy evolution (Xu et al., 2026; Chen et al., 2026a; Lin et al., 2026; Luo et al., 2026; Wang et al., 2026; Chen et al., 2026b). Low behavioral diversity has also been reported for automatically designed agents (El et al., 2025), with conceptual precedents in program equivalence and mutation testing (Budd & Angluin, 1982; Papadakis et al., 2019). Our question concerns attribution in the resulting population: source differences may not change execution, while outcome differences may occur even with identical source. The trace study and clone control examine these two directions separately.

Repeated sampling and ensembles. Pass@ k , self-consistency, and identical-agent ensembles exploit extra attempts (Chen et al., 2021; Wang et al., 2022; Li et al., 2024); even nominally temperature-0 calls can vary (Alizadeh et al., 2023). We do not claim that sampling gains are new. We use them as an explicit reference when evaluating generated-program complementarity. Post-execution verifiers (Kwok et al., 2026) and ensemble selection (Caruana et al., 2004) also have different information from a policy that chooses before any answer is available. Our oracle replay and frozen selector retain this distinction.

Selection and resource allocation. Instance-dependent harness quality (Gupta et al., 2026; Cho, 2026) and HELIX outcome matrices (Fan & Huang, 2026) motivate going beyond the best fixed member. STS, GRASP, counterfactual routing, and RouteLLM select among scaffolds, skills, search actions, or models (Zhou et al., 2026; Moll et al., 2026; Kim, 2026; Ong et al., 2024), extending questions studied in algorithm selection and portfolios (Rice, 1976; Xu et al., 2008; Kotthoff, 2014). The distinction between quantitative and qualitative interaction (Gail & Simon, 1985) explains why

varying effect sizes need not change the best action. We connect task-wise crossovers to a same-code reference and held-out repeats, then evaluate selection separately. This yields a diagnostic contribution, not a new routing algorithm.

3 PROBLEM FORMULATION

A population can supply more correct answers without containing task specialists, and specialists can exist without an effective selector. We formalize these distinctions before describing the controls. Let P contain baseline H_0 , and let $Y(h, x, r) \in \{0, 1\}$ denote member h 's correctness on task x in repeat r . Throughout, gains use % for absolute probability differences: $98.50\% - 93.75\% = 4.75\%$ means 0.0475 additional success probability.

Coverage: were additional correct answers available? A one-run oracle sees correctness before choosing a member. Its headroom over the best fixed member in the recorded matrix is

$$H(P) = \frac{1}{n} \sum_{i=1}^n \max_{h \in P} Y(h, x_i, r) - \max_{h \in P} \frac{1}{n} \sum_{i=1}^n Y(h, x_i, r). \quad (1)$$

This measures available answers, including successes from repeated sampling. The repeat-averaged plugin $\hat{H}$ replaces Y by each member-task mean. With few repeats, maximizing those means can still select favorable estimation errors; $\hat{H} > 0$ is not by itself evidence of specialization.

Stable interaction: does the best member change with the task? To remove execution noise conceptually, write $p_h(x) = \Pr\{Y(h, x, r) = 1\}$ under a fixed execution condition and define $H_{\text{stable}} = \mathbb{E}_x \max_h p_h(x) - \max_h \mathbb{E}_x p_h(x)$. For two members with $\Delta(x) = p_1(x) - p_0(x)$,

$$H_{\text{stable}} = \frac{1}{2} (\mathbb{E}|\Delta(x)| - |\mathbb{E}\Delta(x)|). \quad (2)$$

If one member is better everywhere, the expression is zero, even when the size of its advantage varies. Positive complementarity requires task-wise crossover. Identical programs under the same condition have equal $p_h(x)$ and hence zero stable complementarity. This population identity clarifies the target; it does not estimate unknown probabilities from a small number of executions.

Usable selection: can the advantage inform a choice? A selector π_Z must choose from task features $Z(x)$ before seeing the result. Even stable crossovers may not be predictable from those features. Learning a choice from earlier outcomes on the *same* task instead tests repeat generalization, a different information setting. We evaluate both separately. Practical resource allocation adds one more comparison: a selection gain over one fixed execution must be assessed against additional fixed-program executions under a common budget and answer-selection rule. Equal numbers of harness executions need not entail equal model calls or tokens.

4 EXPERIMENTAL DESIGN

4.1 HARNESS POPULATIONS AND EVALUATION SETUP

A *builder* writes a Python harness; a *target solver* answers its model calls; an external *judge* scores its final output. The solver is GLM-5.3-Flash throughout these studies. Each harness implements `solve(question)` and calls `self.llm`; BIRD also exposes schema and SQL feedback through `self.execute`. Gold answers are available only to the judge. The single-call baseline makes one temperature-0 call. Generated programs can change prompts, decoding, sample count, and control flow, so a fixed solver does not imply identical computation or deterministic outputs. Table 1 identifies the role of each study.

MATH populations and panel selection. Six builders—GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, and ERNIE-5.0-Thinking-Preview—receive eight slots each under one seed. Prompts request an improvement over one greedy call without assigning strategies. Each slot allows three attempts at most (temperature 0.7; 32,768 output tokens per attempt), stopping at the first program that passes parsing, import, interface, and nonempty-answer checks on two

Study	Programs	Evaluation	Question answered
MATH exploration	35 generated + single-call	400 tasks; one run	Is there error coverage?
MATH repeat control	8 generated + single-call, versus 9 clones	Same 400 tasks; 3 fresh repeats	Do choices transfer across repeats?
BIRD discovery	12 generated + single-call + <code>react</code>	60 tasks; one run	What mechanisms execute?
BIRD admission	6 builders $\times$ 3 seeds $\times$ 4 arms	1,169 for primary contrast; 400 for all arms	Does screening change coverage?

Table 1: **Each study addresses a different inference.** One run means one execution per member-task pair. The repeated MATH panel is selected from the exploratory population. BIRD supplies separate execution and admission evidence; effects are not pooled across studies.

development questions. This yields 35 frozen programs from 48 slots; admission does not certify quality or mechanisms.

The repeat panel is drawn without accuracy ranking. A historical mean-call ceiling of 3.05 leaves 21 eligible members; a fixed salted hash selects one per builder and two at-large seats. This favors low-call programs. Selected sources include draft-check-revise, agreement checking, and sample voting; Appendix A.2 lists them. Adding single-call gives nine members. Both this panel and nine identical baseline slots receive three fresh, cache-disabled repeats, with decoding fixed by each member’s source.

MATH-500 (Lightman et al., 2023) is split into 100 development and 400 evaluation problems (fixed random split). Both studies use those same 400 tasks: fresh executions are not unseen-task validation. The judge extracts the final answer, preferring the requested ##### line, then compares numeric values when both parse and normalized LaTeX otherwise. It is neither an LLM judge nor a general symbolic-equivalence checker. Appendix A.1 details the comparator and its audit.

BIRD discovery and admission. BIRD (Li et al., 2023) requires SQL for a question and database. TTHER’s proposer with a GLM-5.3-Flash builder produces the 12 discovery candidates, admitted by import and smoke-execution checks. They and two supplied baselines are evaluated on 60 questions. The separate admission study crosses free versus assigned strategies with ungated versus conformance-gated admission, using six builders, three seeds, and eight slots per arm. At most three attempts per slot are allowed (temperature 0.7; 16,384 output tokens). Assigned strategies include execution repair, three-sample voting, and schema linking. All arms require a valid interface and nonempty SQL; gated arms also check mechanism conformance and exclude some structures.

The admission split reserves 365 questions in two databases for development and 1,169 in nine databases for evaluation; all four arms use a pre-drawn 400-question core. An 18-item smoke test is a disclosed development-exposure exception. Predicted and gold query results are compared using the official set-of-rows scorer with a 30-second timeout and failures scored zero. Discovery instead uses an earlier stringified-row proxy with a row cap. Admission also uses shared response caching, unlike the fresh MATH repeats. These conditions and the bundled nature of the gating intervention are retained in Appendices E.1 and F.

4.2 SAME-CODE CONTROLS AND REPEAT VALIDATION

Same-code slots isolate the sampling explanation. Running more members introduces more attempts as well as more code. We therefore compare nine distinct members with nine byte-identical single-call slots, each evaluated three times per task. The latter preserve the opportunity for stochastic outcomes while removing program specialization. They are an equal-ability reference, not an assumption that all generated programs must behave like single-call.

Held-out repeats separate discovery from validation. Choosing a task’s best program and validating it on the same outcomes would reuse lucky successes. Instead, each of three folds uses two discovery repeats to freeze both a per-task member mapping and a single best fixed member. The third repeat measures their accuracy difference. Averaging folds gives G_{real} and G_{clone} ; the fixed

comparator is discovery-selected, not necessarily single-call. We then compare the arms on their common complete tasks:

$$D = G_{\text{real}} - G_{\text{clone}}. \quad (3)$$

Here the subtraction is of paired task-level gains. It asks whether past outcomes identify choices that transfer to another execution better for distinct programs than for identical ones. Because those past outcomes come from the same evaluation tasks, this is a repeat diagnostic, not an unseen-task router.

Missingness and sensitivity govern the interpretation. A complete task has a verdict for every member and repeat; unknown outcomes stay missing. Descriptive quantities use each arm’s complete tasks, while D uses their intersection. The decision rule requires at most 10% incomplete tasks in either arm and on the intersection, at least 30 paired tasks, and positive-effect checks including $D > 1\%$. Known-truth simulations assess whether the procedure detects crossovers at the available repeat count. These checks distinguish an unresolved result from evidence of absence. Appendix B documents completeness; Appendix C gives the full rule, tie handling, uncertainty, and sensitivity analyses. The acquisition protocol was fixed before collection, with operational amendments during collection; the v3 analysis was revised afterward.

4.3 FROZEN SELECTION AND EXECUTION-MATCHED REPLAY

Feature-based selection before execution. The repeat diagnostic uses past outcomes on the same task; a practical selector must instead choose before seeing an answer. We train π_Z on the 100 development tasks using TF-IDF question features and member accuracies computed within training folds. Five-fold task-level cross-validation keeps all repeats of a task together. The policy is frozen before evaluation and falls back to single-call when its top-two probability gap is below 0.15; evaluation correctness and traces are unavailable to it. Comparators include the development-selected fixed member and a pre-fixed random member.

Oracle replay after execution. A separate offline comparison allocates $b \in \{3, 9, 15, 27\}$ harness executions to a development-ranked portfolio and to the clone arm. Each included member contributes three repeats; the clone reference uses subsets of its 27 recorded executions. Both arms select with post-execution correctness, measuring available oracle coverage. One harness execution can contain several model calls, so equal execution counts need not imply equal calls or token costs. Appendix D specifies the frozen policy, replay, and accounting, separately from the historical archive selection analyses.

5 RESULTS

5.1 SINGLE-RUN COVERAGE IN GENERATED POPULATIONS

We first ask whether the generated population supplies correct answers beyond those of its best fixed member. All 35 admitted generated members are evaluated once on 400 tasks. Their union with single-call reaches 98.50% oracle coverage, versus 93.75% for the best fixed member, single-call. Yet every generated member is individually worse (68.25–91.50%). The resulting 4.75 % headroom is an opportunity available in hindsight, not an observed routing gain.

The outcomes are diverse: 33 of 36 correctness vectors are unique, mean pairwise disagreement is 10.6% (maximum 33.0%), and the union repairs 76.0% of baseline errors. None of these quantities says how to choose a member before execution. This single-run study has no clone control of its own; two reasoning-heavy builders also exhausted generation budgets on most slots, limiting representation. This raises the next question: can additional executions of identical code also yield a positive oracle estimate? The repeat study introduces that control on a selected subset. Its D does not correct the full population’s 4.75 % headroom, and its fresh baseline mean 94.96% is a separate estimate (Appendix A.1).

5.2 POSITIVE ORACLE ESTIMATES FROM IDENTICAL PROGRAMS

The same-code arm provides the most direct check of the specialization interpretation: all nine slots implement the same program, so stable program complementarity is zero. If its estimated headroom is positive, that estimate cannot by itself distinguish specialization from execution variability.

The repeat-averaged estimate is indeed positive: $\hat{H} = 2.09\%$ on 383 complete clone tasks. The generated panel gives 2.38% on its 350 complete tasks (Table 2). These are arm-specific descriptive estimates, not a paired effect comparison. Unknowns remain missing; the older available-repeat estimates use a different estimand and are documented in Appendix B. The control establishes that even an oracle estimate averaged over three repeats can be positive without program specialization. The next question is whether choices discovered in some repeats remain useful in another.

Quantity	Generated panel + single-call	Same-code clones
Complete-task oracle estimate $\hat{H}$	2.38 %	2.09 %
Complete tasks for repeat-based G	350	383
Held-out-repeat mean gain G	0.17 %	0.55 %
Paired clone-adjusted mean: $D = -0.10\%$ on 342 complete tasks		
Incomplete tasks: 12.5% (generated) / 4.2% (clones)		

Table 2: **Positive oracle estimates do not establish a repeat-stable advantage.** Each arm’s $\hat{H}$ and G use its own complete tasks; D averages paired task-level gain differences on the intersection, so subtracting the two displayed G values does not recover D . The generated arm fails the 10% incompleteness gate.

5.3 REPEAT VALIDATION AND DIAGNOSTIC SENSITIVITY

Freezing choices before scoring reduces reuse of chance successes. On the respective complete-task sets, the held-out-repeat mean is $G_{\text{real}} = 0.17\%$ for generated programs and $G_{\text{clone}} = 0.55\%$ for clones. On their 342-task intersection, the calibrated difference is $D = -0.10\%$, with 95% interval $[-1.55, 1.46]\%$. The mean shows no advantage over the same-code reference. It also establishes neither equivalence nor a negative population effect: 12.5% of generated-arm tasks are incomplete, exceeding the 10% gate. The decision is INSUFFICIENT EVIDENCE. All-0/all-1 filling scenarios give G values spanning $[-0.33, 0.33]\%$; the full decision rule and sensitivity methods appear in Appendix C, with missingness records in Appendix B.

Would the diagnostic detect real crossovers? Known-truth simulations answer this necessary follow-up. Across a 100-cell grid varying members, tasks, repeats, and missingness, the clone-calibrated rule’s worst observed null false-support rate is 2%. Sensitivity is much less reassuring: a two-specialist setting with true $H_{\text{stable}} = 17.5\%$ has 0% power at $R = 3$, about 0.7% at $R = 5$, and 100% at $R = 10, 20$. Smaller cyclic crossovers remain largely undetected even at $R = 20$ (Appendix Table 6). These are results for the tested simulation settings, not a universal repeat threshold.

The two findings support different conclusions. The clone result refutes using a positive plugin alone to identify specialization. The repeat result does not settle whether this generated panel has stable crossovers: the data are incomplete and the test can miss known crossovers at three repeats. Some stable mean member differences are detected, but a stronger average member is not itself a task specialist. Although stable specialization remains unresolved, selection performance can still be measured directly. We next use the same MATH panel to test a frozen pre-execution choice and compare post-execution coverage at equal execution counts.

5.4 PRE-EXECUTION SELECTION AND ORACLE REPLAY

Pre-execution selection yields no gain. A frozen feature-based policy tests a choice that can actually be made before execution. On the repeat panel, it selects single-call on all 400 evaluation tasks and ties the development-selected fixed member at 94.96% repeat-mean accuracy: 0.00 % gain. The pre-fixed random-member comparator differs by -10.12% (Appendix D). The policy avoids

weak members but obtains no task-specific benefit. This result concerns one policy and feature set; it neither exhausts possible selectors nor turns the oracle opportunity into achieved accuracy.

Execution-matched oracle replay. Even oracle coverage must be compared with the extra attempts a fixed program could receive. The replay gives a development-ranked panel portfolio and the clone arm the same number b of harness executions. At $b = 3$, the panel contains only single-call; its $+0.37\%$ difference cannot be due to program diversity. Adding members leaves a $+0.03\%$ difference at $b = 9$, followed by negative differences at $b = 15$ and 27 (Table 3).

Harness executions per task b	3	9	15	27
Panel members, including single-call	1	3	5	9
Panel oracle coverage (%)	98.00	98.25	98.25	98.25
Clone oracle coverage (%)	97.63	98.22	98.36	98.50
Panel minus clones (%)	+0.37	+0.03	-0.11	-0.25

Table 3: **The full execution budget does not favor more programs in the observed oracle mean.** Members are ranked on development data and contribute three repeats each. Clone coverage averages subsets of its 27 recorded executions. Both arms use post-execution oracle selection; execution counts are matched, token costs are not.

Collection also costs more for the generated arm: 22,192 logical calls and 54.17M known tokens versus 10,800 calls and 14.51M for clones, a $3.73\times$ token ratio. These are realized collection costs, not a token-matched performance comparison. Cross-arm majority voting is not reported because answer formats are incompatible. Thus the tested selector delivers no gain, and the execution-matched oracle means provide no consistent advantage for this portfolio. Neither result establishes how other selectors or a token-matched operational comparison would perform.

5.5 SUPPORTING ANALYSIS OF EXECUTED BEHAVIOR

The MATH controls show that outcome differences need not require different programs. The supporting BIRD study asks the converse: do source differences produce distinct, useful execution? Here a mechanism is an implemented prompt transformation, tool use, or control-flow operation. This study uses separate populations, judging, and execution protocols; it does not explain the MATH results causally.

Source differences and observed outcomes. All 12 discovery candidates pass a code-difference check, yet 21/66 generated-candidate pairs have identical correctness vectors (36/91 including baselines). Source similarity has essentially no rank association with correctness disagreement ($\rho = -0.010$). Agreement on these 60 tasks is not general program equivalence, but motivates inspecting execution (Appendix Figure 2).

Traces locate three distinct failures. T1: described but absent. Six of twelve candidates claim retrieval or self-checking but make one model call and execute no artifact. **T2: present but untriggered.** On all ten audited `react` tasks, the first SQL executes successfully, so the repair branch never fires and final SQL equals single-call; execution success is not semantic correctness. **T3: executed but broken.** One candidate runs multi-turn repair but returns prose as final SQL, losing tasks that the baseline answers correctly. These failures require implementation checks, branch-exercising tasks, and valid output contracts, respectively. They do not establish that activation improves accuracy. Appendix E retains worked traces, hand-written controls, and their development-exposure limitations.

The separate BIRD admission study also finds a mean headroom contrast of -0.16% for forced/gated versus free/ungated generation. Its gate bundles structural exclusion with conformance verification, so it does not identify a pure verification effect (Appendix F). Source and admission evidence therefore require an execution check; persistent task advantages and selection value remain separate measurements.

6 DISCUSSION AND LIMITATIONS

Implications for evaluation. A positive oracle gain answers whether additional correct answers were available. The same-code control shows why it cannot also answer whether different programs supplied different task abilities. Evaluations should report coverage, repeat-stable task advantages, and realized policy gains as separate results, with a repeated fixed-program reference for resource comparisons. If a gain is attributed to a mechanism, traces must establish what ran and an intervention must test its contribution. This reporting separates useful sampling from specialization without discounting either.

Limitations. The controlled MATH panel is a low-call draw under one target on previously evaluated tasks; its v3 analysis was revised after collection. Missingness exceeds the declared gate, and simulations show that three repeats can miss substantial crossovers. The nonpositive calibrated mean therefore does not establish absence or equivalence. Zero routing gain concerns one selector; the replay matches executions, not tokens. BIRD retains separate single-run protocols, judging, and caching conditions. Its execution failures cannot be used as a causal explanation for MATH.

7 CONCLUSION

Additional coverage, stable task specialization, and achievable selection gains are distinct objects; evidence for one cannot substitute for the others. Here, generated populations supply additional correct answers, but identical programs also yield positive oracle estimates. Missingness and low sensitivity leave stable specialization unresolved. The tested selector gains nothing, and execution-matched replay shows no consistent portfolio advantage. Population evaluation should connect coverage, repeat validation, and actual selection while reporting what each establishes.

ETHICS STATEMENT

All experiments use public BIRD and MATH-500 benchmark data and generated program artifacts; no human participants, private records, or personally identifiable information are collected. Model calls are reported as part of the experimental provenance, and the paper does not release credentials or provider responses that would expose private data. The main ethical risk is overstating what one-run oracle gains imply for deployed routing; we therefore report the unresolved selection and cost claims explicitly as INSUFFICIENT EVIDENCE.

REPRODUCIBILITY STATEMENT

The manuscript specifies the populations, task splits, execution conditions, scoring rules, statistical procedures, and sensitivity analyses used for the reported results. The appendices provide additional protocol details, calibration results, and worked examples. Experimental code is not provided to reviewers; the methodological and empirical claims are presented within the manuscript.

AI USE STATEMENT

This work is, by construction, an AI-assisted research project studying AI-generated artifacts, and we disclose the extent of that assistance precisely.

Required disclosure. We used generative AI tools for: (i) writing and modifying the experimental code (generation pipeline, evaluation collectors, statistical analysis scripts); (ii) running experimental workflows that made model API calls; (iii) analyzing experimental data and producing tables and figures; and (iv) drafting and editing manuscript text. The revised primary and factorial tables and result figure are generated from a shared, versioned reanalysis artifact. Historical exploratory analyses retain their original provenance and stated limitations. The post-review corrections are recorded separately from the original protocol. Verification records specify the scope of automated tests, development controls, and review for each artifact. Post-review revisions also used same-model subagent reviews; these are mechanical checks, not independent cross-model scientific validation. Development controls and local runtime tests do not establish independent gate calibration or provider-level execution independence.

Research ideas and direction. The core research questions, the causal design decisions (estimand, contrasts, exclusion rules), the interpretation standards, and proposed go/no-go criteria were developed with AI assistance. The human authors set the overall research direction and requested the review-driven revision; AI tools also proposed analysis and experiment designs, identified counterexamples, and implemented revisions. Final scientific decisions and approval of the submission remain the authors’ responsibility. This disclosure does not claim that every AI-proposed decision has already received individual author approval.

Responsibility. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI tools. The factual statements in this section are provided for author verification; final responsibility for their accuracy rests with the authors.

REFERENCES

- Vahid Alizadeh et al. An empirical study of the non-determinism of chatgpt in code generation. *arXiv preprint arXiv:2308.02828*, 2023.
- Timothy A. Budd and Dana Angluin. Two notions of correctness and their relation to testing. *Acta Informatica*, 18(1):31–45, 1982.
- Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *Proceedings of the 21st International Conference on Machine Learning*, 2004.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026a.
- Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. HarnessForge: Joint harness and policy evolution for adaptive agent systems. *arXiv preprint arXiv:2606.01779*, 2026b.
- Yong-eun Cho. It’s not the capability: Harness sensitivity is non-monotone across LLM agent tiers. *arXiv preprint arXiv:2605.26731*, 2026.
- Batu El, Mert Yuksekogonul, and James Zou. Inefficiencies of meta agents for agent design. *arXiv preprint arXiv:2510.06711*, 2025.
- Tianyu Fan and Chao Huang. HELIX: Model-harness co-evolution for recursive self-improvement. *arXiv preprint arXiv:2608.13951*, 2026.
- Mitchell H. Gail and Richard Simon. Testing for qualitative interactions between treatment effects and patient subsets. *Biometrics*, 41(2):361–372, 1985. doi: 10.2307/2530862.
- Akshat Gupta, Jermaine Lei, Alexander Lu, Gopala Anumanchipalli, and Leshem Choshen. Automated discovery has no universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- Minho Kim. When should LLMs search? counterfactual supervision for search routing. *arXiv preprint arXiv:2607.05752*, 2026.
- Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(2):48–60, 2014.
- Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. LLM-as-a-verifier: A general-purpose verification framework. *arXiv preprint arXiv:2607.05391*, 2026.

- Jiayi Li, Qin Zhang, Yiting Yu, Qinkai Fu, and Dawei Ye. More agents is all you need. *arXiv preprint arXiv:2402.05120*, 2024.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIRD’s eye view. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Lightman et al. let’s verify step by step appendix: Math-500 subset. *arXiv preprint arXiv:2305.20050*, 2023.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, and Hang Yan. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026.
- Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. Task-CoEvolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*, 2026.
- Johannes Moll, Jean-Philippe Corbeil, Jiazheng Pan, Martin Hadamitzky, Daniel Rueckert, and Lisa Adams. GRASP: Gated regression-aware skill proposer for self-improving LLM agents. *arXiv preprint arXiv:2605.29668*, 2026.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Xinmei Tian, Bo Han, Yike Guo, et al. TTTE: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026.
- Huai Guang Ong, Sylvain Jeremy, et al. RouteLLM: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: an analysis and survey. *Advances in Computers*, 112:275–378, 2019.
- John R. Rice. *The Algorithm Selection Problem*. Elsevier, 1976.
- Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, and Junyao Yang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. doi: 10.48550/arXiv.2203.11171.
- Jinghan Xu, Yikai Zhang, Aili Chen, Weiyuan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026.
- Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, and Zhifei Xie. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution. *arXiv preprint arXiv:2608.25593*, 2026a.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, and Yiqun Zhang. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026b.
- Heng Zhou, Zelin Tan, Zhemeng Zhang, Yutao Fan, Yibing Lin, and Li Kang. Select-then-solve: Paradigm routing as inference-time optimization for LLM agents. *arXiv preprint arXiv:2604.06753*, 2026.

APPENDIX GUIDE

The appendices follow the argument: study objects and protocols, statistical and policy details, supporting BIRD evidence, then historical analyses and deviations.

A Harness Generation, Program Inventory, and Data Splits.

B Same-Code Controls and Repeated-Execution Protocol.

C Statistical Analysis, Calibration, and Sensitivity.

D Frozen Selector and Execution-Matched Replay.

E BIRD Execution Traces and Supporting Analyses.

F BIRD Admission-Policy Study.

G Provenance, Protocol Deviations, and Historical Analyses.

A HARNESS GENERATION, PROGRAM INVENTORY, AND DATA SPLITS

We first document the MATH source population, task splits, and frozen panel, then list model versions. The BIRD discovery and admission protocols are in Appendices E and F.

A.1 EXPLORATORY MATH POPULATION, SPLITS, AND COVERAGE

This single-run study establishes the initial coverage phenomenon and supplies the programs for the repeat panel. It carries the BIRD admission study’s free, ungated generation procedure into competition mathematics, with a domain-specific answer comparator.

Protocol. Six builders (the same models as the BIRD admission study), one seed, eight slots each, an $R=3$ raw-attempt cap per slot, neutral validity gate (mechanism-blind: parse, import, interface, non-empty answer on two dev questions). Frozen target GLM-5.3-Flash (same as the SQL study), with decoding settings defined by each frozen member; the baseline uses temperature 0. MATH-500 (Lightman et al., 2023): 500 tasks frozen before generation (100 dev / 400 eval, fixed random split). The 100 dev tasks are used only for generation-side smoke tests and are never executed in the evaluation archive; the selection-policy experiments of Appendix G.6 therefore carve their training split (80 tasks) from the eval side, leaving 320 scored tasks for D/E claims. This historical archive analysis is distinct from the fresh repeat study, whose selector uses the 100 development tasks and scores all 400 evaluation tasks (Appendix D). Judge: numeric equality where both sides parse as numbers (including a/b , $\frac{a}{b}$, decimals), else normalized LaTeX string equality (units, base subscripts, $^\circ$ stripped; $\texttt{\textbackslash text\{X\}}$ content preserved; $\texttt{pmatrix} \rightarrow \texttt{tuple}$). Judge self-consistency: 400/400 gold answers judge correct against themselves. single-call accuracy 0.9375 under the matched answer-format instruction. Yield: 35/48 slots admitted (R_{artifact} 0.10–1.00; the two reasoning-heavy builders, GLM-5.3 and Qwen3.8-Max, exhausted the 32k-token generation budget on reasoning before emitting code in most slots — reported as-is). This constraint under-represents reasoning-heavy builder styles in the evaluated pool; we do not adjust the budget post hoc, and the limitation is part of the reported protocol. This is a replication probe, not a factorial: no strategy forcing, no conformance gate.

Results (35 harnesses, 400 eval tasks). Mean pairwise outcome disagreement 10.6% (max 33.0%); 33 of the 36 members had unique outcome vectors when single-call is counted (32 of the 35 generated harnesses); outcome-identical pairs 0.7% — no trace of Phase-I-style collapse. Union repair 76.0% of single-call errors; the union of all generated candidates fails on 0.8% of single-call-correct tasks (a *union-all-fail* rate, not an any-candidate or router harm). Observed oracle coverage exceeds single-call by 4.75%, and the best fixed member is *single-call itself* (0.9375; generated harnesses 0.6825–0.915): every generated member is worse individually. The one-run mini-audit originally did not collect repeated-execution or same-code clone controls. The separate WP-1R collection (§5.2) adds those controls on the same task set for eight generated members plus single-call, with nine same-code slots and three fresh repeats. Its acquisition protocol was frozen before collection, while the v3 analysis was revised later. Its nonpositive clone-adjusted mean and limited sensitivity do not retroactively identify the source of the full population’s 4.75% one-run headroom.

A hash-bound re-execution of the versioned endpoint-sensitive rescore of all 14,400 archived verdicts changes 14 of them (0.097%): all are v1 false rejects of interval-endpoint answers on a single task, so per-member accuracies rise by at most one task and no qualitative conclusion moves (Appendix G.6). An earlier stored artifact reporting zero flips is not reproduced by the corrected analysis and is retained only as provenance.

Scope. One seed, one target, no factorial, exploratory; the Phase-I collapse-inducing SQL protocol was not re-run in the math domain, so this replication speaks to the clean-protocol result only, not to the boundary of collapse.

A.2 SELECTED MATH PROGRAM INVENTORY

Table 4 describes the nine frozen members selected for the repeat study. H1–H8 are neutral labels for generated members, all constructed under seed 0. The listed operations describe what each program implements, not whether every branch fires or improves correctness. The mean-call eligibility ceiling is not a per-execution maximum, and one logical call may request several samples.

Frozen member	Source-defined execution path
single-call	One temperature-0 solution with the final-answer format.
H1 (DeepSeek)	Three temperature-0.4 solutions; normalized-answer vote; a greedy fallback when no answer or a tie remains.
H2 (ERNIE)	Draft a solution, then ask the solver to verify/correct the extracted answer; return the checked answer.
H3 (GLM)	Up to five differently prompted temperature-0 solutions; stop when three agree; adjudicate unresolved disagreement, with format fallbacks.
H4 (Kimi)	Request five temperature-0.7 samples in one call; extract answers and vote.
H5 (Kimi)	Initial solution, verification, and an alternative solution; vote, with an additional resolution call if no majority emerges.
H6 (MiniMax)	Draft, critique, and an additional tie-breaking call on disagreement; parse failures use an earlier available answer.
H7 (MiniMax)	Request five temperature-0.7 samples in one call; normalize and vote, resolving ties by first occurrence.
H8 (Qwen)	Draft plus an independent solve-and-audit call; compare answers and request adjudication on conflict, with a format-repair path.

Table 4: **What was selected for the repeat study.** All calls use the same frozen solver; verification and adjudication are solver prompts, not access to benchmark labels. Six seats are builder-stratified; H5 (Kimi) and H7 (MiniMax) are the two at-large seats.

A.3 MODELS AND VERSIONS

Table 5 lists every model used in this paper with its exact version string as sent to the API. All models were accessed through a single commercial OpenAI-compatible API aggregator (the same provider for every role). The frozen target model identity is fixed; decoding follows each harness, with temperature 0 for the single-call baseline. Phase-II harness generation uses temperature 0.7 with a 16,384-token budget. Phase-I generation conditions and their evidence gaps are described in Appendix E.1. Phase I used a proxy execution judge; Phase II uses the official BIRD scorer (30 s timeout), with the legacy judge as a secondary audit. The MATH-500 probe uses the answer comparator in Appendix A.1; no LLM judge produces reported accuracy.

Role	Model (exact version string)	Used in
Frozen target solver	GLM-5.3-Flash (baseline temperature 0)	MATH / BIRD
Second frozen target (replication)	Qwen3.8-Flash	Phase I
Phase-I builders	GLM-5.3-Flash, Qwen3.8-Flash, DeepSeek-V4-Flash-Vision-Exp	§G.1
MATH / BIRD admission builders	GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, ERNIE-5.0-Thinking-Preview	MATH / BIRD
Gate-instrument development	GPT-6	App. F.4
Control authoring	gpt-5.5 (v2/v3 sets), gpt-5.6-sol (v4 set)	App. F.4
Blinded control re-view/adjudication	gpt-5.6-sol (v2/v3), gpt-5.5 (v4, semantic adjudication)	App. F.4
Judge	proxy / official BIRD execution scorer; MATH answer comparator	see text

Table 5: Complete model inventory. The target is held frozen within each phase; Phase-II builders come from six independent labs.

B SAME-CODE CONTROLS AND REPEATED-EXECUTION PROTOCOL

This section records acquisition, complete-task estimands, and missing outcomes. Statistical decisions and calibration are in Appendix C; the frozen selector and replay are in Appendix D.

Frozen protocol. The acquisition design was fixed before collection. Statistical and operational specifications were amended before confirmatory outcomes; the attempt ceiling increased from 45,000 to 60,000 and concurrency from 2 to 6 during collection. Eligibility requires historical mean logical calls ≤ 3.05 , favoring low-call mechanisms rather than a uniform sample of the 35 generated members. A fixed salted-hash draw selects one member per builder and two at-large seats. Both the nine-member panel and nine byte-identical single-call slots receive $R = 3$ fresh repeats. The fixed MATH-500 split contains 400 evaluation and 100 development tasks. Evaluation tasks were used in the earlier one-run study; the new collection validates fresh executions, not unseen tasks.

Descriptive estimates and paired inference. The complete-task oracle estimates are 2.38 % for the 350 complete generated-arm tasks and 2.09 % for 383 complete clone tasks. The earlier values of 2.54 % and 2.67 % use available-repeat means over all 400 tasks, filling fully unobserved means with zero. These are different descriptive estimands. The main text uses complete tasks; the paired D uses their 342-task intersection and remains INSUFFICIENT EVIDENCE.

Collection interruptions and missing outcomes. Collection used a 60,000-attempt global budget with per-cell atomic reservation. Two provider-timeout interruptions split the generated arm across three continuation groups and the clone arm across two. The solver condition was held fixed across groups. A completed observation takes precedence over an unresolved request; among completed records for the same cell, the first is retained. Across 19,318 generated-arm records, 8,418 were migration copies. Completed observations replaced unresolved markers 14 times; 9 earlier unresolved cells were later completed. There were 5 cells with disagreeing completed executions; the primary analysis retains the first and a sensitivity analysis excludes them. The clone arm’s 14,967 records reduce to 10,800 unique cells, with 4,160 copies and 0 conflicts.

72 cells remain unresolved and missing from the outcome matrices. The primary G uses complete tasks without imputation. Sensitivity scenarios fill unknown outcomes with all zeros or all ones and recompute the full cross-fit, including selection changes. Three collector versions served the generated-arm continuations with concurrency 2/2/6. Program identity and solver conditions were checked at combination; these operational differences are retained as study conditions.

C STATISTICAL ANALYSIS, CALIBRATION, AND SENSITIVITY

This section expands the held-out-repeat diagnostic in the main text. Historical archive applications and their separate data splits are retained in Appendix G.6.

C.1 DECISION RULE AND POST-COLLECTION ANALYSIS REVISIONS

The primary complementarity statistic is the cross-fitted frozen-selection gain G of §4.2: in each of three folds, two repeats freeze the per-task member mapping and the best *fixed* member, the third repeat scores the frozen mapping against that fixed member; G is the per-task mean averaged over folds. Tie-breaking is label-symmetric: because $R=2$ discovery scores lie on $\{0, \frac{1}{2}, 1\}$ and ties dominate at high accuracy, a deterministic lexicographic rule would bias the selection toward the first listed member, so iid $U(0, 10^{-9})$ jitter is drawn per (fold, task) and averaged over ten frozen tie-realizations; a simulation over dominant-member positions verifies the statistic is order-independent. The decision statistic is the real-minus-clone difference $D = G_{\text{real}} - G_{\text{clone}}$ on the task intersection complete in both arms. Its reference distribution uses 10,000 two-arm randomizations: within every task and repeat, member labels are permuted independently in each arm and D is recomputed (a single permutation joint across repeats was rejected in development because it preserves a genuine crossover mapping and does not null it). A paired task bootstrap (2,000 draws with a fixed random seed) gives the 95% interval. The frozen rule emits SUPPORTED only when the coverage gate passes ($\leq 10\%$ incomplete tasks in either arm and on their intersection; ≥ 30 paired tasks), the randomization $p < 0.05$, the bootstrap lower bound is positive, $D > 1.0\%$, and G_{real} 's own lower bound is positive; REFUTED requires the D upper bound within the margin and a small equivalence p ; otherwise INSUFFICIENT. The acceptance rule that any of these states terminates the study without re-running remains. The earlier v2 procedure — a discovery-repeat argmax scored against single-call on repeats 2–3 (A8.6) — is retained as a labeled secondary statistic, not a complementarity test (under pure global dominance with single-call at 0.5 and the other members at 0.9 its expectation is $\approx 0.5(1 - 0.1^8) \cdot 0.4 \approx 0.20$). Two frozen sensitivity branches accompany it (within-task available-repeat mean imputation; exclusion of doubly-executed conflict cells).

C.2 DIAGNOSTIC TARGETS AND CALIBRATION

For a population of harnesses and a single-call reference, the diagnostics pipeline evaluates five classes of claims on an explicit execution matrix $Y[h, x, r, c]$ (member, task, repeat, execution condition): (A) measurement sufficiency (manifest consistency, judge replay with an explicit executed/not-executed state, identity uniqueness; a check that was not run is reported as not-run and can never render the class SUPPORTED); (B) single-shot coverage (oracle minus best-fixed, with dev-fixed reported separately); (C) stability, in two separated estimands; (D) pre-execution selectability (a frozen one-call selection policy versus dev-fixed); (E) budget-matched policy utility under a frozen 1-call budget with $\lambda=1$ harm weighting (offline, one-call; E leaves INSUFFICIENT_EVIDENCE unless the per-task call budget is verified by per-record evidence or is part of the generating design). Each class returns SUPPORTED, REFUTED_WITHIN_SCOPE, INSUFFICIENT_EVIDENCE, or NOT_APPLICABLE with scope and minimal missing evidence; classes are never collapsed into a single score.

Two stability estimands. *C-rank*: pairwise same-condition accuracy differences with a SHARED task-resample bootstrap and max-T simultaneous 95% intervals over all $M(M-1)/2$ canonical member pairs (family-wise error control; the pre-v3 per-pair intervals did not adjust for 36 simultaneous comparisons), plus sign consistency across repeats; the pair direction is fixed by member identity, so population conclusions are invariant to member ordering. The population state means *at least one* pair differs after multiplicity adjustment, not that the complete ranking is stable. *C-comp*: for a general population stable complementarity is $H_{\text{stable}} = \mathbb{E} \max_h q_h(x) - \max_h \mathbb{E} q_h(x)$, zero exactly when one member is expectation-best on every task; for two members this reduces to the half-absolute-value form of Eq. (2), which is the only form displayed in the main text. Repeats with differing execution-condition fields or differing program identities are refused as matched repeats and force INSUFFICIENT_EVIDENCE; byte-identical outcome matrices are refused unless both repeats carry distinct, explicitly independent execution provenance (legitimate near-deterministic programs can reproduce identical matrices, so identity equality triggers provenance verification rather

than an automatic rejection). Pair-level imputation bounds over unknown cells are reported alongside the recorded success rate.

The revised C-comp decision. The repeat-averaged plug-in of H_{stable} is retained as a *descriptive* field only; it never determines a scientific state. Two known-truth counterexamples from the review are encoded as regression tests: all $q = 0.9$ with $M=9, T=400, R=3$ gives a plug-in $\hat{H} = 8.69\%$ (the mean over the 300 fixed simulated datasets), and the earlier available-repeat clone estimate is 2.67% (the complete-task estimand is reported in Appendix B) — both with true $H_{\text{stable}} = 0$. The calibrated statistic is the cross-fitted frozen-selection gain G described in §4.2; the decision uses the real-minus-clone difference D .

What holds at the population level, and what is checked empirically. Let θ^* be a selector’s population gain relative to the population-best fixed member. On every task the selected member’s probability is at most $\max_h q_h(x)$, so $\theta^* \leq H_{\text{stable}}$. The null $H_{\text{stable}} = 0$ is the global-dominance cone: some h_0 satisfies $q_{h_0}(x) \geq q_h(x)$ for all h and almost every x . Under dominance a population-optimal mapping routes to the dominant member, so the population gain is $\theta^* \leq 0$ throughout the cone, not just at equal ability. This compares with the *population-best fixed* member, whereas the finite- (R, T) statistic G uses a fixed member selected from discovery data. In that statistic, the discovery argmax is selected from noisy repeats, and a weaker member that ties the noisy discovery score can be chosen in a finite sample (the sealed clone arm prints $G_{\text{clone}} = 0.55\%$ under finite-sample variation), so $\theta^* \leq 0$ does not by itself bound the finite-sample rejection probability. The two-arm Monte Carlo randomization reference is exact at the equal-ability boundary, where permuted labels are exchangeable and the recomputed statistic (with the +1 correction) is a draw from the same conditional distribution. Over the rest of the dominance cone, false support is verified *empirically* on every null DGP in the calibration grid below — including heterogeneous difficulty, heterogeneous dominance, the 0.5/0.9 configuration, missingness, and sticky within-task latent states matched between arms — rather than asserted as an analytic finite-sample result. A level-.05 procedure can false-support at level .05 on a finite grid; the defensible claim is the observed worst rate and its Wilson interval, not impossibility.

Tie-breaking. Member-label tie-breaking in the frozen argmax is iid $U(0, 10^{-9})$ jitter averaged over ten frozen tie realizations on the real-data report (each calibration dataset uses one draw: under the null the tie bias has mean zero); a deterministic lexicographic rule is not label-invariant and is rejected by simulation. Label-symmetry is asserted directly by a permutation-invariance regression test: the 10-draw tie-averaged G , D , and state agree up to Monte Carlo tolerance when the member order is permuted identically in both arms.

Randomization mechanics. Labels are permuted independently within every task AND every repeat (a single joint permutation across repeats preserves a genuine crossover mapping and does not null the statistic: verified in development), recomputing both G ’s, their difference, and the two selections on every draw; the +1 Monte Carlo correction gives an exact discrete level. A paired task bootstrap supplies the sampling interval; a $\leq 10\%$ incomplete-task coverage gate, extremal unknown-cell bounds, and a positive G_{real} sign guard complete the rule, and without a clone arm no SUPPORTED state is emitted.

regime	$R=3$		$R=10$		$R=20$		abst.
	plug	v3	plug	v3	plug	v3	$R=3$
equal $q=0.9^\dagger$	100	0	100	0	100	1	100
equal + difficulty [†]	100	0	100	1	100	1	100
0.5/0.9 dominance [†]	100	0	100	0	13	0	100
dominance [†]	100	0	100	0	100	0	100
cross $H \approx 0.9\%$	100	1	100	0	100	0	99
cross $H \approx 1.8\%$	100	1	100	1	100	0	99
cross $H \approx 4.2\%$	100	0	100	0	100	0	100
two strong specialists, $H=17.5\%$	100	0	100	100	100	100	100

[†]Null ($H=0$): false-support %, nominal 5%; else power %. $M=9$, $T=400$, $B=300$ at $R=3$ (150 sweeps), 300 randomizations. Wilson intervals and the full M , T , R , missingness grid in Appendix C.

Table 6: Three repeats can miss even a strong positive case. At $M = 9$, $T = 400$, null rows report false-support % and alternative rows report power %; the final column gives $R = 3$ abstention. The clone-calibrated rule (v3) rarely supports nulls, but has little sensitivity to the tested small and moderate crossovers.

Known-truth simulation calibration. Binary tensors are drawn from specified $M \times T$ probability matrices: iid equal ability at constant q and with shared task difficulty; global dominance with a clean dominant fixed member, the reviewer’s 0.5/0.9 configuration, and heterogeneous dominance; and crossover alternatives of increasing strength over two latent task types, plus difficulty-related partial crossover. The grid spans $M \in \{3, 9, 20\}$, $T \in \{100, 400, 1000\}$, $R \in \{3, 5, 10, 20\}$, complete data, 5% MCAR, and 3% hardness-related missingness; the paper table focuses on the $(9, 400, \cdot)$ acquisition design with 300 Monte Carlo datasets per cell (150 in sweeps). Every cell reports the support, refutation, abstention and error rates of four procedures (plug-in; one-split frozen gain; repeat-cross-fitted gain; clone-calibrated v3 decision), interval coverage against the true parameter, and mean estimates. The plug-in frequently false-supports under the tested nulls; v3’s worst observed null support rate is 2%. Its power reaches 100% for the strong two-specialist alternative at $R = 10, 20$, while the smaller cyclic crossovers remain largely undetected even at $R = 20$. These empirical rates characterize the tested grid, not a general size or power guarantee. The earlier eight mechanism-derived classes and the four repeat-based classes (C9–C12, with C9/C10 now paired with registered same-code clone arms) continue to match their predefined expected states; invariant tests additionally assert permutation invariance, dominance $\neq$ complementarity, identity refusal, and the two review counterexamples directly.

C.3 FINITE-REPEAT BIAS AND CLONE-NUL CALIBRATION

Writing $\hat{q}_h(x)$ for a repeat-averaged estimate, $\mathbb{E} \max_h \hat{q}_h(x) \geq \max_h \mathbb{E} \hat{q}_h(x)$ by Jensen for each task, with strict inequality when finite- R noise can change the per-task argmax. The plug-in *headroom* is a difference of two such maxima; Jensen bounds each term separately and therefore does NOT establish that the difference is upward biased in general — in a two-member, two-task crossing example the plug-in can be downward biased (true $H=0.40$, expected plug-in over three repeats ≈ 0.34). What is established is stronger and narrower: under same-code exchangeability the true H_{stable} is zero while the plug-in is positive with probability approaching one (selection noise on the per-task argmax; $\hat{H}=8.69\%$ in the $q=0.9$, $M=9$, $T=400$, $R=3$ known-truth simulation, and 2.09 % descriptively on the sealed clone arm’s 383 complete tasks; the real panel gives 2.38 % on its 350 complete tasks). They use the same complete-task sets as each arm’s G , but are not the paired comparison used for D . The calibrated procedure never decides from the plug-in: a discovery/validation cross-fit freezes the per-task member mapping on discovery repeats and scores it on held-out repeats against the best *fixed* member selected on the discovery repeats. Finite discovery data need not identify a globally dominant member. The real-minus-clone difference uses the clone arm as an observed sampling reference; it is not an analytic removal of all finite- (R, T) selection bias. Inference uses a two-arm Monte Carlo randomization reference (+1 correction; exact at the equal-ability boundary, empirically checked across the dominance grid rather than asserted analytically throughout the cone; Appendix C). The identity gates handle the degenerate case outright by requiring provenance-verified independent executions; identical matrices without distinct provenance are refused.

C.4 VALIDATION ON KNOWN-TRUTH CONTROLS

The frozen package has two layers: the original eight mechanism-derived classes $\times$ 2 instances (16 controls, expectations unchanged), plus four repeat-based classes $\times$ 2 instances exercising the stability paths single-shot controls cannot reach — global dominance (C-rank SUPPORTED, C-comp REFUTED), equal-mean crossover skill (C-rank INSUFFICIENT, C-comp SUPPORTED under clone calibration, and the visible stratum makes D/E SUPPORTED), a condition-mixing trap, and a cloned-repeats trap (both must abstain); C9/C10 carry registered same-code clone arms as required by the v3 decision. Expected states are fixed before any blinded run and derive from the generating mechanisms. Invariant tests additionally assert permutation invariance, dominance $\neq$ complementarity, identity refusal, the two review counterexamples (iid noise and 0.5/0.9 dominance must not support; crossover must support with adequate repeats), and that the decision never supports without a clone arm. Calibration matched on all state cells; the blinded set (12 controls + 2 auditor challenges re-executed from the pre-committed, fixed specification under the current code) matched on all 14×6 cells: zero false supports, the positive control recognized, and the abstention cap holds. These discrete controls complement, but do not replace, the Monte Carlo false-support/power calibration of §5.3, which reports the v3 test’s empirical false-support rates and power on the tested grid. The earlier stored challenge result whose CH2 numbers contradict its own spec (best-fixed 0.94 is unreachable under the generating rule, which forces 0.79) is retained as provenance only; spec-faithful re-execution matches the auditor’s pre-committed expectations on every cell.

D FROZEN SELECTOR AND EXECUTION-MATCHED REPLAY

This section concerns the fresh MATH repeat study: the selector is fitted on 100 development tasks and scored on 400 evaluation tasks. The historical 80/320 archive split and BIRD database-split selection analyses are separate studies in Appendices G.6 and G.4.

D.1 FROZEN FEATURE SELECTOR

π_Z is multinomial logistic regression over TF-IDF question features (max 300) and per-member dev accuracies computed within training folds only; 5-fold task-level CV on the 100 dev tasks with same-task repeats never split across folds; fold-model averaging; ties broken by member id; abstain to single-call when the top-2 probability gap is below 0.15. Repeat aggregation uses the available-repeat mean per member-task (same-task repeats stay in one fold). Comparators: single-call, dev-fixed, pre-fixed random member (seeded), M1b (dev-fixed with single-call fallback on missing cells). E_1 (budget by construction) is not applicable because panel members make more than one call; E_b with $b=3$ calls per task is reported as a separately named estimand. Leakage controls: no answer, correctness, eval-derived difficulty, post-execution trace, or judge feature is used.

Policy	Eval acc. (%)	Δ vs dev-fixed (%) [95% CI]
single-call (fixed)	94.96	0.00 [0.00, 0.00]
dev-fixed	94.96	—
random (pre-fixed)	85.14	-10.12 [-13.27, -7.22]
M1b (dev-fixed + bare fallback)	94.96	0.00 [0.00, 0.00]
π_Z (frozen selector)	94.96	0.00 [0.00, 0.00]

Table 7: **WP-2R frozen comparator set on the 400 MATH-500 eval tasks** (repeat-mean correctness over three fresh repeats; retrospective tasks, prospective execution — no unseen-task claim). Paired differences are per-task against dev-fixed with task-level bootstrap CIs.

D.2 DIAGNOSTIC DEFINITIONS OF SELECTION AND BUDGET UTILITY

Class D evaluates one frozen deterministic policy $\pi_Z : Z(x) \mapsto h$ — pre-execution features (question text, frozen mechanism labels, training-fold dev accuracies), fitted on development tasks only — by $D(\pi_Z) = \mathbb{E}_x[Y(x, \pi_Z(x)) - Y(x, h_{\text{dev-fixed}})]$, with an abstention rule falling back to single-call when the top-2 selection-probability gap is below the frozen margin. Class E evaluates

$U_b(\pi) = \mathbb{E}_x[\Delta(x, \pi(x))]$ under a uniform per-task call budget b with $\lambda=1$ harm weighting (one regression cancels one repair); it is admissible only when per-record evidence verifies the budget on every evaluated member-task pair, and abstains otherwise.

D.3 EXECUTION-MATCHED REPLAY AND REALIZED COLLECTION ACCOUNTING

Three distinct comparisons are reported and never conflated. (i) *Equal-slot collection*: both arms execute nine members/slots for three repeats; this is the randomization design and does not imply matched spend. (ii) *Common per-task execution budget* (§5.4): offline comparison at $b \in \{3, 9, 15, 27\}$ executions per task between the dev-ranked panel portfolio and b of the 27 same-code executions, under post-execution *oracle* aggregation only; panel-minus-same-code oracle differences are $+0.37, +0.03, -0.11, -0.25\%$. No cross-arm majority-vote accuracy is reported because the arms emit incompatible answer formats (boxed numbers versus free-form prose), so a modal answer key would be unfair. (iii) *Realized spend*: per-request accounting layers experiment-run > harness-execution > logical-call > provider-attempt; migration copies are excluded. The real arm made 22,192 logical calls (32,097 provider attempts; 54.17 M known tokens; 54.5 calls per eval task) versus the clone arm’s 10,800 logical calls (10,804 attempts; 14.51 M tokens; exactly 27 per task) — a $2.05\times$ call and $3.73\times$ token ratio driven by the multi-call panel members; the conservative ceiling at the frozen break-even price is 1,803.82 vs 483.15 CNY (pricing not independently verifiable; reported as token counts times the frozen 33.3 CNY per 1M tokens, never as billed amounts). Totals across all arms: 38,458 logical calls, 50,817 attempts, 79.8 M known tokens, 2,656.22 CNY ceiling. We therefore make no realized-cost-matched claim; the common-budget replay matches harness executions, not token cost.

E BIRD EXECUTION TRACES AND SUPPORTING ANALYSES

This appendix expands the supporting execution diagnosis. In the historical records, Phase I denotes BIRD discovery and Phase II the separate BIRD admission study; neither explains the MATH results causally.

E.1 DISCOVERY POPULATIONS AND CONDITIONS

Discovery status. Phase I (§5.5 and G.5) is a discovery study. 24 of its 151 evaluation items overlapped protocol-development measurements; the human control was designed after inspecting discovery traces (its observed headroom is descriptive, not a blind estimate of stable complementarity). Phase-I admission thresholds (union repair $\geq 15\%$, headroom $\geq 5\%$, ≥ 3 pairwise-distinct non-empty fix sets) were fixed before the 151-item evaluation; the v6→v7 threshold change (20%→15%) is disclosed, and both readings admit the same populations. This is an internal pre-evaluation freeze, not external preregistration.

Data hygiene. The single-call baseline flipped 3 of 60 items between the two discovery runs (API nondeterminism at temperature 0), so 60-item columns are conditioned on slightly different single-call-error sets. Phase-I judging used a set-of-rows-equality proxy that we later measured to be uniformly conservative relative to the official BIRD scorer (on the two Phase-I matrices with logged final SQL, it never credits an answer the official scorer rejects; agreement 0.97–0.99).

Phase-I populations. Seed baselines: single-call (single call) and *react* (execution feedback loop, shipped with TTHE). *Old proposer protocol*: six candidates selected from TTHE’s optimize-loop output (GLM-5.3-Flash builder), selected for behavioral diversity using only the 36 questions disjoint from the evaluation set, by a rule frozen before evaluation. *Strategy-forced free-form*: GLM-5.3-Flash, Qwen3.8-Flash, and DeepSeek-V4-Flash-Vision-Exp builders, eight assigned mechanisms each; the GLM population is the logged re-run ($K=3$; the original run recorded zero accepted outputs but its tool was not preserved and the observation is unaudited). *Harness IR*: six harnesses compiled deterministically from declarative mechanism specifications (context $\in \{\text{plain, schema.link, hint.guard, link+hint}\}$, candidate generation with temperature, per-candidate execution, repair rounds, selection rule), so a declared mechanism is present in the control flow by construction. *Human positive control*: the four hand-written harnesses of Appendix E.2.

Generation-log gaps and historical protocol probes are retained in Appendix G.



Figure 2: **Source differences need not change observed correctness.** BIRD discovery: 14 harnesses, 60 tasks, one execution each. Across 91 pairs including baselines, token-sequence source similarity has $\rho = -0.010$ rank association with correctness disagreement; 36/91 pairs have identical correctness vectors. This is an outcome comparison on the observed tasks, not a test of program equivalence.

E.2 DISCOVERY MEASUREMENTS AND HAND-WRITTEN CONTROLS

Every BIRD discovery candidate passes a code-difference check. Yet 21/66 generated-candidate pairs have identical correctness vectors (36/91 including baselines), and source similarity has essentially no rank association with correctness disagreement across the 91 pairs ($\rho = -0.010$; Figure 2). Mean disagreement is 3.3% overall (3.8% among generated candidates), union repair of single-call errors is 7.7%, and one-run headroom is 3.3%. Agreement on these 60 tasks does not establish general program equivalence, but it motivates examining what the programs actually do.

E.3 TRACES DISTINGUISH ABSENCE, NON-ACTIVATION, AND FAILURE

T1: described but absent. Six of twelve candidates claim retrieval or self-checking in docstrings but make a single model call and execute no artifact. The named mechanism is not implemented. **T2: present but untriggered.** In all ten audited `react` tasks, the first SQL executes successfully, so an existing `execute-feedback-repair` branch never fires; final SQL equals single-call. Successful execution is not semantic correctness, and these runs do not test repair under failure. **T3: executed but broken.** One of twelve candidates performs multi-turn execution and repair but returns prose as final SQL, losing tasks that single-call answers correctly.

These observations locate different failures rather than assigning one diversity score. T1 requires checking implementation; T2 requires tasks that exercise the branch; T3 requires a valid output contract. They do not establish that activation causes improved accuracy. Four hand-written controls with distinct paths produce 6.67% one-run headroom on 60 tasks and 7.28% on 151 items, showing that the apparatus can record different repairs. But they were written after trace inspection and lack matched repeats and clone controls. Appendix E.1 discloses the legacy proxy and 24/151 items overlapping protocol development; Appendix E.4 gives worked traces.

E.4 WORKED EXAMPLES FROM THE TRACE AUDIT

To make the failure taxonomy of §5.5 concrete, this section walks one BIRD task through the audited harnesses. Outputs are quoted from the discovery trace audit (5 harnesses $\times$ 10 tasks); its logger caps stored SQL excerpts at 120 characters, so truncations are marked *[trunc.]*.

The task. BIRD dev-set question 352 (`card_games` database; question text verbatim, the typo is the dataset’s):

Calculate the percentage of the cards available in Chinese Simplified.

Its BIRD hint:

‘Chinese Simplified’ is the language; percentage =
Divide(Sum(id where language = ‘Chinese Simplified’),
Count(id)) * 100

Gold SQL (executing the two on the released database yields 8.7734 and 8.7728 respectively — the official scorer separates them):

```
SELECT CAST(SUM(CASE WHEN T2.language = 'Chinese Simplified'
THEN 1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id) FROM cards
AS T1 INNER JOIN foreign_data AS T2 ON T1.uuid = T2.uuid
```

The single-call harness answers:

```
SELECT SUM(CASE WHEN language = 'Chinese Simplified'
THEN 1 ELSE 0 END) * 100.0 / COUNT(id) AS percentage FROM
foreign_data;
```

marked wrong by the official scorer: it counts directly over `foreign_data`, whereas the gold query counts after joining to `cards`; the hint does not spell out that join.

react (T2: mechanism present but untriggered). `react`’s control flow does execute SQL and retries on database errors. But its first call is byte-identical to single-call’s prompt, and here the generated SQL executes without error, so the error-triggered retry never fires and the final SQL equals single-call’s. Across all 10 audited tasks the pattern held: SQL executions present, final SQL byte-identical to single-call’s on 10/10 — the feedback loop is real code whose repair branch did not activate on these tasks. Controlled failure injection is needed to test whether feedback changes the output.

Single-call candidate (T1: mechanism described, not executed). This candidate’s docstring claims a “conservative repair”: cap the solver’s maximum output tokens and demand an answer-only block. Its behavior is structurally single-call’s: one LLM call followed by SQL extraction. The token cap exists only as an environment default (2,048 against the runtime’s 32,000) — no harness control flow implements it, and it never binds on these tasks because a SQL answer is orders of magnitude below the cap. On the worked task it issues exactly one call, executes nothing, and returns the same wrong query as single-call modulo whitespace (counting over `foreign_data` without the join). Across the 10 audited tasks it records zero SQL executions on all 10, and its final SQL is byte-identical to single-call’s on 6 (the remaining differences trace to the reworded system prompt, not to any mechanism). The most common Phase-I failure class is therefore not broken machinery but absent machinery: the claimed mechanism exists only in prose.

Timeout-retry candidate (code-level diversity, behavioral identity). This candidate is a genuinely different program: it wraps the single coder call in a daemon-thread watchdog and fires one hedged duplicate re-ask if the call stalls past 40 s. The mechanism exists in the control flow — but its trigger (a stalled API call) never occurred in the audit: zero SQL executions, and the final SQL equaled single-call’s on all 10 audited tasks. Code difference without behavioral difference survives even when the added mechanism is real: it simply never fires.

Multi-turn repair candidate (T3: a broken multi-turn path). The one audited candidate implementing genuine execute-inspect-repair returns the model’s *prose* as final SQL. On a task single-call answered correctly (“State the alternative languages available for card named Annul numbered 29.”), its logged final SQL begins, verbatim:

with "only a single read-only SELECT is allowed". The previous "query" was actually just prose text (the model wrote `rea[trunc.]`)

The repair re-prompt invites the failure: asked to fix a query that "was actually just prose", the frozen target responds with more prose reasoning, no SQL block is found, and the reasoning text is submitted as the query. The item flips from single-call-correct to harness-wrong.

E.5 ARCHIVED BIRD EXECUTION FINGERPRINTS

This fingerprint analysis is post-hoc on the archived confirmatory cells (18 builder $\times$ seed cells, 384 admitted candidates, core-400 outcomes). It uses the main reanalysis's record-combination rules, fixed tasks, and member identity checks. These tasks had already been analyzed, so the results are descriptive, not independent transfer validation. Historical selection on the same archive is in Appendix G.4.

Rich fingerprints. To isolate the loader change, the W3 reanalysis retains the legacy lossy SQL text transform: semicolon deletion, backtick replacement, lowercasing even segments after splitting on single quotes, and whitespace joining. This is not a SQL parser or semantic normalization; it can alter literals and quoted identifiers. Among 364,428 pair-task observations with two identical, nonempty transformed SQL strings, 0 have different verdicts. This is an observed consistency check, not a property guaranteed by construction.

Pair-level SQL dissimilarity averages (1-token-Jaccard) only over verdict-agreeing tasks with two nonempty SQL strings. Its Spearman correlation with overall outcome disagreement is 0.597 across 4,386 within-cell pairs and 0.588 across 1,010 within-arm pairs. Pairs share candidates and tasks, and the SQL metric conditions on verdict agreement; these correlations have no independence-based significance or causal interpretation.

In R2's 70-harness first cache-off acquisition, call counts change on 506/28000 comparisons and verdicts on 2391/28000. Transformed SQL changes on 19569/27936 comparisons (70.05%); 64 comparisons lack two nonempty SQL strings and are excluded only from the SQL denominator. The mean token Jaccard among changed SQL pairs is 0.619. These channel-specific rates do not define an effect-size noise floor. Canonical logical-call costs are reported separately in Table 12.

E.6 SECOND-TARGET PROBE

A second-target probe using Qwen3.8-Flash reproduced outcome collapse. The strategy-forced population retained headroom, while the prompt-side human control did not transfer wholesale. These exploratory observations do not establish target-independent specialization.

Which mechanism hypothesis follows from the traces? The untriggered BIRD branch suggests comparing repair enabled, repair disabled, and repeated single-call on clean SQL tasks and controlled execution failures. Repeated, budget-matched gains where feedback changes the query and restores correctness would support a repair mechanism; equally effective resampling would weaken it. This intervention has not been run. It tests a specific repair hypothesis, not a universal necessity claim for harness diversity.

F BIRD ADMISSION-POLICY STUDY

The study uses a 2 $\times$ 2 factorial design plus an open-mechanism arm, with six builders and three seeds. Each slot allows at most $R = 3$ attempts, stopping at first admission without top-up. Two development databases are excluded from the evaluation pool. The evaluation set contains 1,169 questions from nine databases and a pre-drawn 400-item core. An 18-item pipeline smoke test is a disclosed exception to development isolation. The official BIRD judge enforces a 30 s timeout. Responses are cached by model identity with cross-process locking; a cache-concurrency defect found during collection was corrected.

The original primary contrast was D–A on paired builder–seed cells with hierarchical bootstrap and permutation inference, with Holm-corrected factorial secondary analyses on the common core

and K-matched sensitivity analyses. Instrument-calibration fixes preceded confirmatory outcomes. Other deviations included operational access handling and quarantine of inconsistent observations. The corrected analysis uses shared baseline records and database/task resampling; these revisions followed the original analysis specification. The implemented gate structurally rejects two permitted prompt-level strategies, and realized generation costs differ across arms.

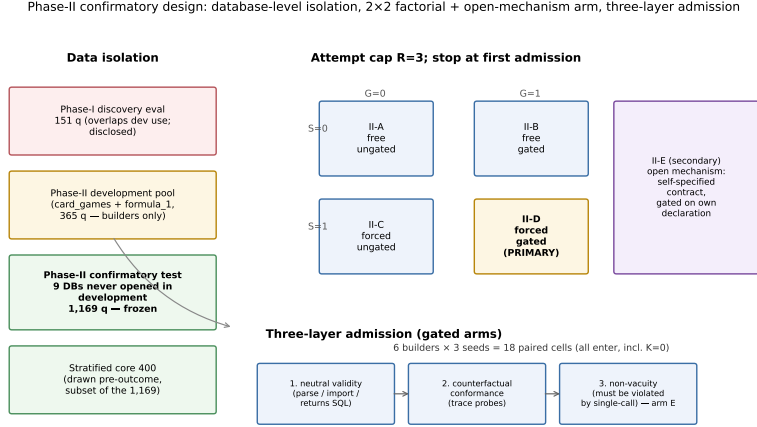


Figure 3: **Phase-II confirmatory design** (frozen protocol). Left: database-level contamination isolation. Middle: the 2x2 factorial with a common attempt cap, plus open-mechanism arm II-E. Right: the three-layer admission pipeline.

F.1 DESIGN AND PRIMARY CONTRASTS

A separate confirmatory BIRD study crosses six independent builders with three seeds under strategy forcing (a named mechanism from a fixed set versus free invention) and conformance gating, giving four arms: II-A (free, ungated), II-B (free, gated), II-C (forced, ungated), and II-D (forced, gated). The evaluation covers 1,169 questions from nine databases never opened during development (one disclosed 18-item smoke-test exception), with the official BIRD execution scorer and a pre-outcome-frozen analysis later re-run as a versioned corrected reanalysis. The implemented gate bundles nontrivial-mechanism structural screening with conformance verification — two of eight forced prompt-transformation slots cannot pass even when neutrally valid — so the treatment does not isolate verification causally.

Metric	II-A	II-D
Candidate count	5.78	3.83
Oracle accuracy (%)	76.14	75.70
Best-fixed accuracy (%)	67.58	67.30
Headroom (%)	8.56	8.40
D—A headroom (%)	−0.16	
Unadjusted bootstrap 95% CI	[−0.83, +0.87]	
One-sided sign-flip <i>p</i>	0.647	

Table 8: **Corrected primary analysis.** Means over 18 builder–seed cells on 1,169 questions; single-call is included in both the oracle and best-fixed comparators.

The diagonal forced/gated minus free/ungated contrast in single-call-inclusive oracle headroom is −0.16 %, with cell contrasts ranging over −4.36 to + 2.74 %: the bundled screening policy does not improve mean headroom. The original uncertainty analysis is retained in Table 8; this mean comparison does not establish equivalence. Structural admission is therefore not evidence that a mechanism activates on relevant tasks or adds stable utility.

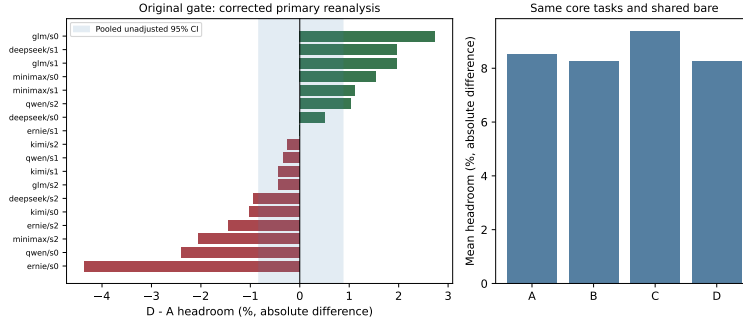


Figure 4: **Versioned corrected reanalysis.** Left: full-set D–A cell contrasts with the pooled 95% interval. Right: four-arm headroom on the shared core-400 tasks with a common single-call baseline.

On the shared core-400 tasks the gate contrast is -0.69% , and the strategy and interaction secondaries are likewise inconclusive (Table 9); the fixed- K mean contrast -0.31% provides a population-size sensitivity check. The open-mechanism arm ran only on the development side and has no confirmatory estimand. These are all single-run contrasts, and the execution variability reported in the R2/R3 audits in Appendix F.3 (8.54% R2 verdict flips; 14.5% of tasks changing single-call correctness across three runs; a 6.50% same-code one-run gap) characterizes them as coverage opportunities rather than capability differences.

Contrast	Effect (%)	95% CI (%)	Raw p	Holm p
Gate	-0.69	$[-1.56, +0.20]$	0.026	0.079
Strategy	$+0.42$	$[-0.32, +1.51]$	0.263	0.428
Interaction	-0.87	$[-2.83, +0.65]$	0.214	0.428

Table 9: **Corrected core-400 factorial reanalysis.** Gate is $[(B-A) + (D-C)]/2$, strategy $[(C-A) + (D-B)]/2$, interaction $(D-C) - (B-A)$; raw and Holm-adjusted p -values.

F.2 SAME-CORE HEADROOM DECOMPOSITION

Table 10 separates coverage, best-fixed accuracy, and headroom using the same core tasks and baseline records in every arm.

Arm	K_{cand}	K_{total}	Single-call (%)	Best (%)	Oracle (%)	H (%)
A	5.78	6.78	63.25	66.00	74.51	8.51
B	6.11	7.11	63.25	66.12	74.39	8.26
C	5.61	6.61	63.25	65.64	75.01	9.38
D	3.83	4.83	63.25	65.65	73.90	8.25

Table 10: **Same-core headroom decomposition.** All arms use identical 400 tasks and identical single-call outcomes. K_{cand} excludes single-call and $K_{\text{total}} = K_{\text{cand}} + 1$; counts and accuracies are cell means. Headroom is oracle minus best-fixed accuracy, not realized routing gain.

F.3 SENSITIVITY, REPEAT AUDITS, AND RECORDED COST

Corrected K-matched and R2 replays. Both replays use the same deterministic record ordering and shared database/within-database task draws as the corrected primary analysis, with seeds re-sampled within fixed builders (10,000 replicates). For K-matching, each A/B and C/D pair uses the smaller candidate count; every uniform candidate subset is enumerated exactly. Single-call stays in every subset, and best-fixed is recomputed inside each subset on every task draw. This is a size sensitivity analysis, not a causal mediation decomposition. Retaining all 18 cells, including two with singleton A subsets, changes subset D–A from $+0.42$ to $+0.81\%$ (change $+0.39\%$, 95% CI $[-0.95,$

+1.63)). R2 subset headroom excludes single-call because R2 did not acquire a matched single-call run, so it is distinct from the single-call-inclusive primary estimand; the old 16-cell comparison is reconciled here. These are instability measurements, not standard errors or a detectability floor for the mean treatment effect; they do not establish that every contrast is unchanged, or identify what fraction of between-harness disagreement is noise.

R2 acquired 70 harnesses on 400 tasks, selecting the first two callable admitted candidates per A/D cell where available. The two cache-off acquisition groups are kept separate despite a duplicated repeat label; program identity is checked against the cached runs. No matched single-call was acquired, so R2 headroom excludes single-call in both conditions. The corrected analysis retains singleton cells (their subset headroom is zero), yielding all 18 paired cells. There are 2391/28,000 verdict flips against cached outcomes (8.54%) and 1245/14,400 flips between the two D cache-off passes (8.65%). For reconciliation only, restricting to the 16 cells with at least two members in both arms gives D−A −0.11 % cached and +0.36 % cache-off. This explains the difference from the full-cell estimates; neither analysis identifies future execution variance, the effect of caching alone, or a stable-complementarity advantage over clones. Acquisition time and caching condition are not isolated.

R3 single-call-only repeat audit. R3 is a separate, smaller instability audit: three independent cache-off executions of the *single-call baseline only* (no generated harnesses) on the same core-400 tasks, same frozen target (GLM-5.3-Flash, temperature 0). 58 of 400 tasks (14.5%) changed single-call correctness in at least one of the three repeats; pairwise flip rates between repeat pairs are 39/400, 40/400, and 37/400 (9.75%, 10.0%, 9.25%). Treating the three single-call repeats as three pseudo-members (one execution each) gives a one-run oracle of 72.25% against a best-single-repeat accuracy of 65.75%, a descriptive 6.50 % same-code oracle gap computed from the recorded official-scorer outcomes. R3 therefore measures the observed instability rate of the baseline itself under temperature-0 API nondeterminism. This is not a standard error or an effect-size floor for any contrast, but it sets the scale at which repeat controls become necessary for interpretation: a harness-versus-single-call difference of this magnitude cannot be attributed to capability without them.

Sensitivity estimand	Estimate (%)	95% CI (%)
K-matched B−A headroom	−0.55	[−1.53, +0.63]
K-matched D−C headroom	−0.08	[−1.26, +0.80]
K-matched gate mean	−0.31	[−1.07, +0.40]
R2 subset D−A (cached)	+0.42	[−0.61, +1.48]
R2 subset D−A (cache-off)	+0.81	[−0.35, +1.84]
R2 change in subset D−A	+0.39	[−0.95, +1.63]

Table 11: Corrected sensitivities on core-400, 18 cells. CIs are unadjusted. K-matched comparisons always retain single-call; R2 comparisons exclude single-call and are conditional on the observed acquisition groups.

Recorded cost and missing billing data. The same canonical core records contain 259,304 logged logical solver calls across 153,600 candidate–task records (Table 12). These calls include cache-served requests and are not provider-billed API requests. None of the inspected records has token usage fields; we do not impute token or monetary expenditure from call counts. Actual generation attempts, admitted candidate counts, and these execution counts therefore remain separate measurements. The population cost mean gives each builder–seed cell equal weight and excludes single-call; zero-candidate populations would contribute zero solver calls.

Arm	Candidate-task records	Logged solver calls	Calls/task/population
A	41,600	67,258	9.34
B	44,000	64,399	8.94
C	40,400	75,604	10.50
D	27,600	52,043	7.23

Table 12: Descriptive canonical cost audit on core-400, 18 cells per arm. Logged logical calls are not billable requests or token counts. The last column averages candidate-population calls per task over cells.

F.4 INDEPENDENT CONTROLS FOR THE ADMISSION INSTRUMENT

We tested the post-review Gate-v2 development instrument with 32 newly authored controls, separate from the controls used to develop its probes. The instrument was fixed before a separate model (gpt-5.5; Table 5) received the public contracts and callable interface, without its implementation or earlier controls. The author supplied two intended conforming and two intended nonconforming implementations for each of the eight mechanism types. Control source, author-reference labels, instrument, measurement code, and execution order were fixed before the first evaluation. No benchmark or provider generation was run, and this first measurement is retained unchanged.

Declared mechanism	Structural pass	Structural fail	Review required
Repair	2	2	0
Execution-result vote	2	2	0
Schema linking	0	4	0
Hint guard	0	0	4
Two views	0	3	1
Decomposition	1	3	0
Error classification	0	4	0
Format guard	0	0	4
Total	5	18	9

Table 13: First evaluation of 32 independently authored controls by the fixed development instrument. No case had an execution failure. These are instrument states, not correctness labels or population error rates; the author’s intended positive/negative labels require separate scrutiny.

The nine review-required cases were retained. A third model (gpt-5.6-sol) received the full control source and traces without author-reference labels or automatic verdicts, for source-contract review of all 32 controls and semantic adjudication of the four designated mechanism types. Independent source review supported conformance for one decomposition control and two error-classification controls that the instrument rejected. The decomposition stub returned the payload of the first plan step whose text occurred in a request; retaining the earlier step in the context of the next request therefore retrieved the earlier payload again. The classification check removed the exact error text but still counted an ordinary `Schema: heading` as a category, making the extracted category set disagree with the expected error class. These are instrument failures on these controls, not observed failures of a target model. Three author-intended positive controls were disputed: two schema-linking controls retained an unrelated table, and one two-view control batched its requests and mishandled an empty result. Their original labels and the review disagreements were retained separately. Among the 29 controls whose author labels and source review agreed, the instrument rejected 3 of 13 conforming controls; all 16 nonconforming controls were rejected after the designated semantic review. The instrument therefore did not pass this finite control check. These counts do not estimate error rates in a generated population. Semantic adjudication of the designated 16 controls yielded 7 conforming, 8 nonconforming, and 1 indeterminate case. The indeterminate case already had a structural failure, which was retained without resolving its semantics. This small, deliberately balanced control set is not a random sample of generated harnesses. Passing it would not validate the production interface or replace the common-raw-pool comparison of admission policies.

Revised instrument and a new independent control set. Gate-v3 corrected decomposition request attribution and treated category keywords and shared request templates as review clues rather than automatic rejections. We fixed this version before fresh model contexts (gpt-5.5 authoring, gpt-5.6-sol review) authored and reviewed a new set of 32 controls, again two intended positive and two intended negative controls per mechanism. The review packet withheld author labels, automatic verdicts, and instrument-derived attribution metadata. All 32 controls received source review; the 20 controls in the four prompt/formulation groups plus decomposition also received semantic or data-flow review. Structural outcomes were 8 passes, 10 failures, and 14 review-required cases, with no execution failures. Following one recorded evidence clarification, source review classified 15 controls as conforming and 17 as nonconforming; semantic review classified 8 as conforming, 11 as nonconforming, and 1 as indeterminate.

The indeterminate classification control requested an intermediate error category, but the synthetic interface returned SQL text. Its fallback then selected a semantics repair action. This establishes a response-type mismatch in the fixture, not a real solver’s classification error. A second classification control was reference-disputed: its source mapped the supplied schema-failure message to a semantics action. Among the 31 undisputed references, 14 of 15 conforming controls were accepted and one remained unresolved; all 16 nonconforming controls were rejected. The prespecified finite check therefore remained unmet because of the unresolved case and insufficient agreed positive coverage for error classification. One voting control’s conformance applied only to result arbitration; it did not establish task-conditioned candidate generation. These finite mechanism checks do not validate complete harness operation or replace the planned common-pool policy comparison.

Runtime response contracts. We subsequently tested seven archived ungated, strategy-forced candidates, selecting the first available candidate per strategy in a fixed ordering, rather than by performance; no format-guard candidate was available (Appendix G.2). The 27 synthetic scenarios exposed both candidate and instrument limitations. The two-view candidate skipped its second execution when the first result was nonempty, contrary to its original “execute both” declaration. In contrast, the schema-linking candidate requested JSON but received plain text, and the full plan in decomposition requests prevented the frozen probe from uniquely attributing the current target. We retained those observations and tested source-assigned response profiles in 11 further development scenarios. Dedicated subquestion fields identified the current target, executable SQL payloads reached the final assembly, and four errors produced by an in-memory SQLite database elicited the corresponding syntax or schema repair instructions. Schema-linking JSON changed the explicit table/column restrictions in the SQL-writing request, although the full schema remained present. Retaining schema context therefore cannot by itself establish that the linked subset was ignored. These exposed-candidate checks distinguish response-interface failures from observable candidate deviations; they do not replace independent calibration or provide revised benchmark outcomes.

Unified interface and premeasurement reference review. Gate-v4 combined the source-assigned response interfaces with the five unchanged Gate-v3 mechanism probes. A separately authored set of 32 controls (gpt-5.6-sol) underwent blinded source review before execution. The author was familiar with earlier control failures; schema-response and declared voting-parameter mismatches found in this source review were corrected, with the initial versions retained. The revised references had two agreed conforming and two agreed nonconforming controls per mechanism. After freezing their source, profiles, and dependencies, one measurement covered 124 scenarios without execution exceptions. Structural results were 4 passes, 6 failures, and 22 review-required cases.

A new review context (gpt-5.5) received all source and actual traces without reference labels, source-review decisions, or automatic verdicts. Its first judgments were 18 conforming and 14 nonconforming. Retaining structural failures in the prespecified hybrid rule accepted all 16 conforming references but also one nonconforming hint control, so the finite zero-error criterion was not met. That control correctly restated existing hints but invented a mandatory sort order when no hint was supplied. A recorded, targeted evidence clarification corrected this oversight and a two-view judgment that had treated two samples from one request as independent formulations. The latter control was already rejected structurally. The clarified decisions agreed with all 32 references, but this follow-up does not replace the first blinded result or constitute a new independent calibration pass. No instrument rerun, provider generation, or benchmark evaluation was performed in this check.

Scope of the declaration vocabulary. These controls cover the eight strategy-forced contracts, not the free arm’s four declarations (repair, vote, twostage, and plain). In the original free prompt, vote specifies multiple samples and selection but not three-way majority voting; twostage specifies artifact consumption without a particular response interface. The plain declaration also specifies no control-flow mechanism, whereas the free prompt requests a control-flow change. Applying the eight-strategy gate to these declarations therefore requires separately fixed semantic rules and an indeterminate policy, rather than silently treating vote as vote3 or choosing a compatible profile after observing its result. This source-level scope gap does not estimate false-rejection rates for generated candidates.

Upstream extraction issues and incomplete provider acquisition records are retained in Appendix G.

G PROVENANCE, PROTOCOL DEVIATIONS, AND HISTORICAL ANALYSES

This section preserves historical analyses, protocol probes, and provenance gaps without treating them as confirmatory results of the fresh repeat study. MATH collection amendments and missingness remain in Appendix B, analysis revisions in Appendix C, and admission-design limitations in Appendix F.

G.1 EARLY GENERATION-PROTOCOL PROBES AND THEIR LIMITS

Phase I also probed generation-side fixes — a strategy-forced free-form protocol (assigned mechanism, import/smoke acceptance) and a compiled Harness IR. Under the strategy-forced protocol two of three builders produced populations whose point estimates exceeded the pre-specified thresholds (Qwen3.8-Flash 5.96, DeepSeek-V4-Flash-Vision-Exp 7.95 %; the old-protocol arm missed at 4.64 %), and per-strategy acceptance ordered with builder identity (GLM-5.3-Flash 3/8, Qwen3.8-Flash 7/8, DeepSeek-V4-Flash-Vision-Exp 8/8, under the logged tool). These observations motivate Phase II but are not confirmatory, for three disclosed reasons: the design was two nested contrasts that cannot separate protocol from builder main effects; the early runs lack per-attempt logs, so budget equality cannot be audited; and acceptance was an import/smoke check, not the counterfactual conformance gate of §F.1. Full condition details are in Appendix E.1.

Generation-null failures (M0). Re-running the original optimize loop with Qwen3.8-Flash and DeepSeek-V4-Flash-Vision-Exp builders produced 40 candidates byte-identical to the single-call baseline source. Code identity alone does not imply identical stochastic executions. This run supplies no distinct-code population; it is reported as a distinct, more primitive failure mode — the pipeline produced no distinct population at all. The collapse claims above rest exclusively on the genuinely distinct-source discovery population.

Figure data. Figure 2 compares source similarity with correctness disagreement for the observed harness pairs.

Generation-provenance gaps. The early free-form populations were produced in-session; their generation tool was not preserved, and per-attempt logs do not exist for them — so budget equality cannot be audited for those runs, and we do not claim it. Their acceptance was an import/smoke check at evaluation time, *not* the Phase-II counterfactual conformance gate. The subsequent generation procedure records every attempt (3 retries, 16,384-token budget, per-attempt stage and finish reason); the completed per-strategy grids and the builder-acceptance ordering (3/8, 7/8, 8/8) come from this logged tool and are post-hoc with respect to the frozen admission decisions.

G.2 UPSTREAM CODE-EXTRACTION AUDIT

Across the 36 forced C/D cells, all 108 `format_guard` attempts failed neutral validity: 71 logs report unterminated Python strings, 36 report no Python fence, and one reports a provider timeout. The archived generator’s non-greedy Python-fence extractor terminates at the first subsequent triple backtick, including one inside a Python string. Three synthetic, syntactically valid programs containing SQL fences in an ordinary string, docstring, or regex become invalid prefixes; the extractor

nevertheless sets its truncation flag to false. The audited extractor matches the retained implementation, but the historical attempt records do not verify which version actually ran. Full builder messages were not saved by this generator. Thus the failure pattern is compatible with extraction damage, not proof that all historical failures had this cause or that repaired candidates would pass admission. We found 74 retained extracted programs, all syntactically invalid; three coexist with no-fence logs and are retained as provenance conflicts, not attributed to those attempts. These findings place a generation interface failure alongside the gate’s explicit strategy exclusions.

G.3 POST-REVIEW PROVIDER ACQUISITION STATUS

The post-review provider acquisition attempts did not yield a complete common pool: transport interruptions left completed responses, unclosed requests, and unstarted attempts in separate auditable states. These records are retained for provenance and missingness accounting, but are excluded from claims about population performance, cost, or stable complementarity. Thus, the manuscript does not use provider availability or partial acquisition as evidence for the unresolved clone-control question.

G.4 HISTORICAL BIRD SELECTION ANALYSES

This selection analysis and the fingerprints in Appendix E.5 are post-hoc on the archived confirmatory cells (18 builder×seed cells, 384 admitted candidates, core-400 outcomes). Both use the same record-combination rules, fixed tasks, and member identity checks as the main reanalysis. They remain descriptive: these tasks had already been analyzed, and this is not independent transfer validation.

Diversity-aware selection. Each cell pools all admitted candidates across arms. The reanalysis retains the legacy seed, 100 random database-half splits, 200 random subset draws per split and cell, and candidate-selection rules. It records a new deterministic schedule: the historical set-derived cell order was not archived, so identical historical splits and draws cannot be recovered from the seed alone. Selection uses development databases; evaluation uses the complementary databases. The overlapping splits are not 1,800 independent experiments. Methods are top- k development accuracy, uniform random subsets, greedy development oracle coverage, and greedy mean accuracy plus λ times oracle coverage for $\lambda \in \{0.25, 0.5, 1, 2\}$. All candidate choices are frozen within a split before its evaluation.

Table 14 retains the common AD single-call in every evaluated subset; k counts candidates only. Candidate-only values are recorded separately to distinguish the historical estimand. Oracle and best-fixed use held-out outcomes as descriptive upper bounds. Dev-fixed instead chooses one member using development accuracy, with ascending member-name ties and single-call first. Deterministic methods already retain a globally dev-best candidate; differences in this comparator may arise from which tied maxima survive, not improved maximum development accuracy.

At $k = 8$, div-only versus top-acc changes oracle by +0.35 %, best-fixed by −0.42 %, and headroom by +0.78 %. Dev-fixed changes by −0.001 %. These are distinct quantities, and none is an independently validated routing gain. We withdraw the legacy cell-only confidence intervals; the current table is descriptive while inference covering database/task sampling and re-selection remains outstanding. The earlier normalized-headroom ratio is excluded from the reported analysis.

k	Selection	Oracle (%)	Best (%)	H (%)	Dev-fixed (%)
4	top-acc	73.98	66.64	7.34	65.10
4	random	73.90	66.03	7.87	64.66
4	div-only	74.22	66.18	8.04	65.13
8	top-acc	76.03	67.11	8.93	65.10
8	random	76.15	66.76	9.39	64.95
8	div-only	76.39	66.68	9.70	65.10

Table 14: Canonical descriptive W1 reanalysis. Every pool contains k selected candidates plus the common single-call. Means over 18 cells and 100 overlapping database splits; no confidence intervals. Dev-fixed is chosen only on development data; Best uses held-out outcomes.

Re-selection does not establish interval coverage. An exact counterexample rules out a universal 95% coverage guarantee for our proposed positive-weight re-selection percentile interval. Let nine independent databases each contain one Bernoulli(p) task state. Sixteen fixed candidates comprise eight state-0 specialists followed by eight state-1 specialists; single-call is always wrong. Apply the canonical $k = 8$ rules to four development and five held-out databases. With the recorded tie rules, the expected div-only minus top-acc headroom statistic is $(1 - p^4) \mathbb{E}[\min(B, 5 - B)/5] > 0$, where $B \sim \text{Binomial}(5, p)$. On the event that all nine states coincide, every positive reweighting produces zero headroom for both methods, so the percentile interval is $[0, 0]$. Coverage is therefore at most $1 - p^9 - (1 - p)^9$, or 61.26% at $p = 0.1$. Averaging identical cells or overlapping fixed splits does not remove this event. This bound concerns the stated construction and interval, not BIRD-specific coverage or the impossibility of other inference methods.

G.5 ROUTING AND EXPLORATORY SELECTION

The Phase-I leave-one-harness-out predictor did not beat the best fixed harness in the tested DeepSeek population (-3.4%); the Qwen estimate was $+0.9\%$ (CI $[-5.2, +6.9]$), and task-held-out repair AUROC was 0.512. These predictor and split-specific results do not establish value prediction. The canonical W1 analysis is likewise descriptive: at $k = 8$, diversity versus top-accuracy selection changes oracle by $+0.35\%$, best-fixed by -0.42% , and headroom by $+0.78\%$; the development-selected fixed member changes by -0.001% . None is a validated routing gain (Appendix G.4).

G.6 HISTORICAL ARCHIVE DIAGNOSTICS AND RESCORING

Applied to the MATH-500 archive (36 members $\times$ 400 tasks) and the 18-cell BIRD Phase-II archive. On BIRD, the integrity class (A) is REFUTED_WITHIN_SCOPE for every cell: the combined observations contain overlapping acquisition records that duplicate on the canonical key (resolved by the archived first-write-wins policy, with recorded verdict-conflict counts), so the coverage numbers below are computed on that deduplicated view, not on a conflict-free archive; judge replay is not executed on this path and is reported as not-run, not passed. Subject to that scope, all cells show single-shot coverage (B SUPPORTED); stability, selectability, and deployability are INSUFFICIENT_EVIDENCE by the frozen rules because the archives contain no matched repeats and no dev/eval split for policy training — not because costs are absent: per-record logical calls are archived and are read on the same canonical deduplicated view as the rest of the reanalysis (259,304 calls across 153,600 candidate-task core-subset records, Table 12), though token/dollar costs remain unrecoverable. On MATH the frozen selection policy, trained on the first 80 of the 400 eval tasks held out of all D/E claims (the archive contains no dev-set executions: the 100-task dev split of Appendix A.1 was used only for generation smoke tests, so the policy split is carved from the eval side, leaving 320 scored tasks), ties with dev-fixed (single-call) on every eval task, so selectability and deployability are INSUFFICIENT_EVIDENCE rather than refuted. Three published numbers were corrected (membership-basis of the outcome-vector count, now 33/36 members including single-call and 32/35 generated harnesses; the generated-harness accuracy range 0.6825–0.915, previously printed as 0.68–0.87; the headroom decomposition with best-fixed = single-call stated explicitly), two overloaded metric names were disambiguated (union-all-fail vs any-candidate-fail), and one preserved-judgment control confirms a reported pair (union repair 76.0% / union-all-fail 0.8%) under the explicit definition. A re-execution of the versioned endpoint-sensitive judge over all 14,400 MATH verdicts changes 14 of them — all v1 false rejects of interval-endpoint answers on one task; the v1 replay itself is internally consistent, and an earlier stored 0-flip artifact is not reproduced by the corrected analysis and is superseded. The bracket-merging defect the new judge was designed to catch therefore does fire in this archive, on exactly one task.

G.7 QUARANTINED MATH PROMPT-PROTOCOL PILOT

First, the clean-protocol diversity result replicates in a second domain with a different output space. Second, a quarantined pilot round supplies an exploratory comparison: when the same six builders were (mistakenly) given a GSM8K-style task description whose numeric answer format did not match MATH-500, the generated population remained behaviorally diverse (16.2% disagreement, 19 unique outcome vectors) while its union repair collapsed to 18% — and code-level inspection of the catastrophic members reproduced the Phase-I trace taxonomy verbatim (a generate-then-verify

harness whose verifier rejected nearly every answer and whose fallback returned the stale loop variable or “0”). Diversity without utility, induced by a prompt-side protocol error invisible to code-level checks: the measurement mandate, not any code-level audit, is what detects it.

G.8 PARALLEL DOMAIN EVIDENCE FOR THE MEASUREMENT BOUNDARY

Table 15 places the two domains side by side for the same diagnostic question, without pooling them. MATH-500 reproduces the boundary: one-run disagreement and 4.75 % headroom coexist with single-call as the best fixed member (93.75%). The one-run probe itself has no repaired-gate control; clone, repeat, and selector controls come from the separate fresh WP-1R collection (§5.2).

Domain / analysis	n_H	n_{eval}	Disagree.	Oracle	Best-fixed	H (%)
BIRD, A / D	18 cells	1,169	—	76.14 / 75.70%	67.58 / 67.30%	8.56 / 8.40
BIRD, D—A	18 paired	1,169	—	—	—	−0.16
MATH-500, free	35	400	10.6%	98.50%	93.75% (single-call)	4.75

Table 15: Parallel domain evidence for the same measurement question. BIRD is the archived factorial analysis; MATH-500 is a one-seed exploratory free-arm probe. Designs are not pooled. Headroom is a one-run diagnostic, not selector accuracy or stable complementarity.